

Data Science II Homework 5

Yongyan Liu (yl6107)

2026-04-30

Problem 1

In this problem, we will apply support vector machines to predict whether a given car gets high or low gas mileage based on the dataset `auto.csv` (used in Homework 3). The response variable is `mpg_cat`. The predictors are `cylinders`, `displacement`, `horsepower`, `weight`, `acceleration`, `year`, and `origin`. Split the dataset into two parts: training data (70%) and test data (30%).

```
auto <- read.csv("auto.csv") %>%
  dplyr::select(mpg_cat, cylinders, displacement, horsepower,
               weight, acceleration, year, origin) %>%
  mutate(mpg_cat = factor(mpg_cat, levels = c("low", "high")))

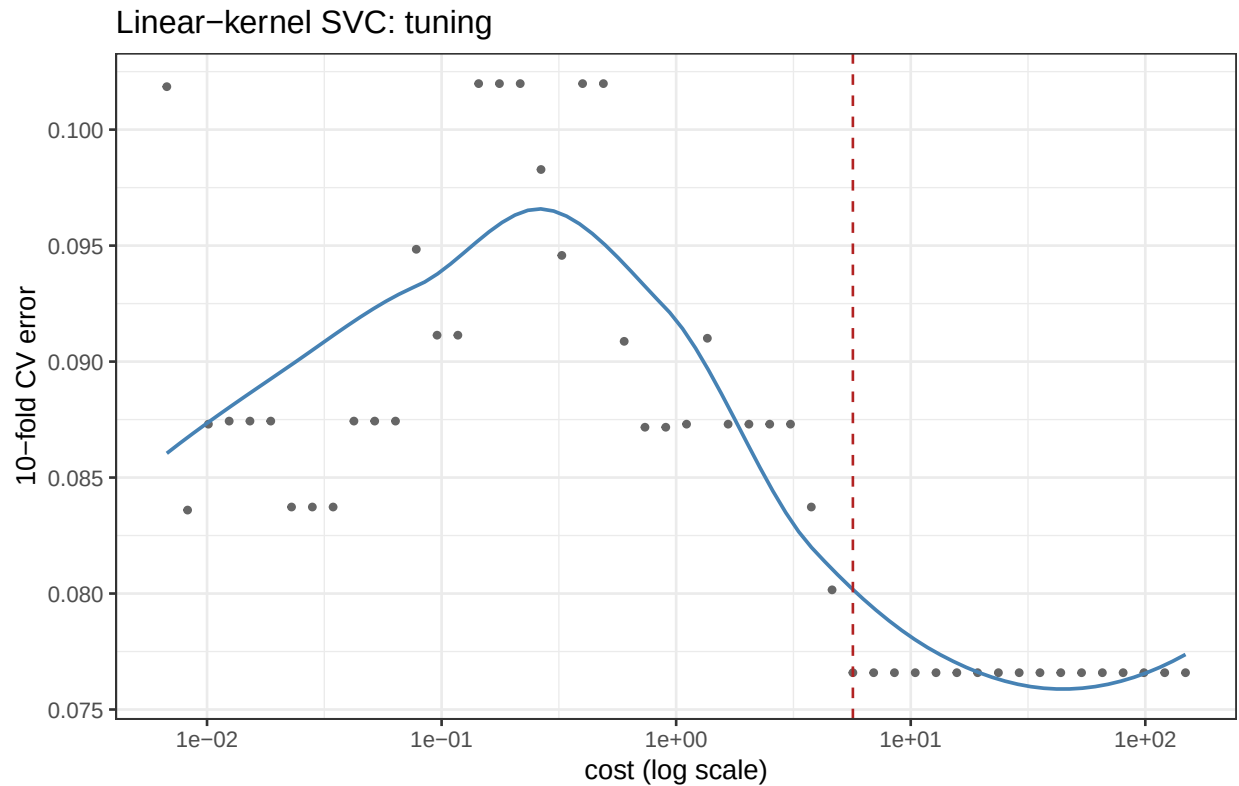
set.seed(2026)
split1 <- initial_split(auto, prop = 0.7)
train1 <- training(split1)
test1 <- testing(split1)
```

(a) Fit a support vector classifier (SVC) to the training data. What are the training and test error rates?

We use `e1071::tune.svm` with a linear kernel and tune `cost` over a wide log-spaced grid via 10-fold cross-validation, mirroring the Lecture 13 “Linear boundary” pipeline. The grid is extended to `exp(5)` so that the selected cost falls in the interior rather than at a boundary.

```
set.seed(2026)
linear.tune <- tune.svm(mpg_cat ~ . ,
                      data = train1,
                      kernel = "linear",
                      cost = exp(seq(-5, 5, len = 50)),
                      scale = TRUE)

ggplot(linear.tune$performances, aes(x = cost, y = error)) +
  geom_point(size = 1, colour = "grey40") +
  geom_smooth(se = FALSE, colour = "steelblue", linewidth = 0.7) +
  geom_vline(xintercept = linear.tune$best.parameters$cost,
            linetype = "dashed", colour = "firebrick") +
  scale_x_log10() +
  labs(x = "cost (log scale)", y = "10-fold CV error",
       title = "Linear-kernel SVC: tuning") +
  theme_bw()
```



```
best.linear <- linear.tune$best.model

pred.linear.train <- predict(best.linear, newdata = train1)
pred.linear.test  <- predict(best.linear, newdata = test1)

train_err_svc <- mean(pred.linear.train != train1$mpg_cat)
test_err_svc  <- mean(pred.linear.test  != test1$mpg_cat)

confusionMatrix(data = pred.linear.test, reference = test1$mpg_cat)$table
```

```
##           Reference
## Prediction low high
##      low   51    4
##      high   8   55
```

The CV-selected cost is 5.6672. The linear SVC achieves a **training error rate of 0.073** and a **test error rate of 0.102**.

(b) Fit a support vector machine (SVM) with a radial kernel to the training data. What are the training and test error rates?

We tune `cost` and `gamma` jointly via `e1071::tune.svm` over a 2-D grid (mirroring the Lecture 13 “Radial kernel” grid). The `cost` lower bound is extended to $\exp(-3)$ and the `gamma` upper bound to $\exp(2)$ so that the CV-selected pair sits well inside the grid rather than near a boundary.

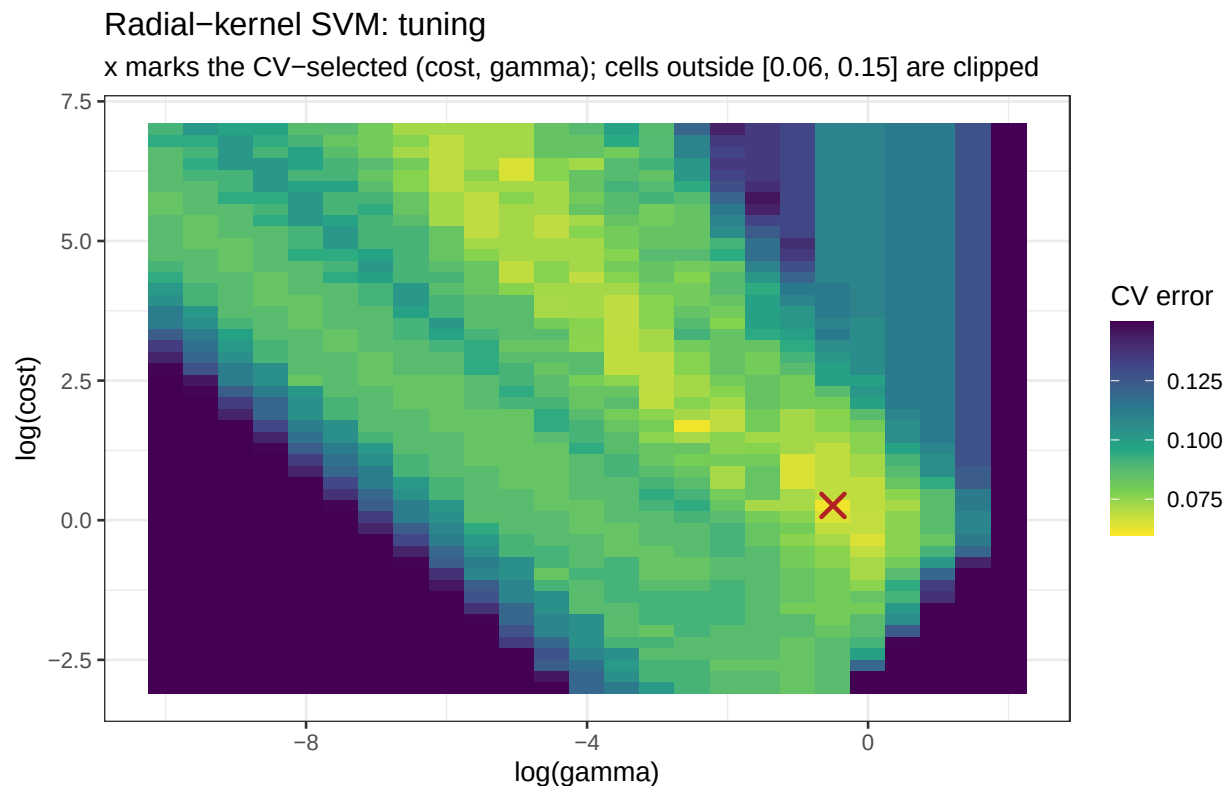
```

set.seed(2026)
radial.tune <- tune.svm(mpg_cat ~ . ,
  data = train1,
  kernel = "radial",
  cost = exp(seq(-3, 7, len = 50)),
  gamma = exp(seq(-10, 2, len = 25)))

best_pt <- radial.tune$best.parameters

ggplot(radial.tune$performances,
  aes(x = log(gamma), y = log(cost), fill = error)) +
  geom_tile() +
  scale_fill_viridis_c(direction = -1, name = "CV error",
    limits = c(0.06, 0.15),
    oob = scales::squish) +
  annotate("point", x = log(best_pt$gamma), y = log(best_pt$cost),
    shape = 4, size = 3.5, stroke = 1.3, colour = "firebrick") +
  labs(x = "log(gamma)", y = "log(cost)",
    title = "Radial-kernel SVM: tuning",
    subtitle = "x marks the CV-selected (cost, gamma); cells outside [0.06, 0.15] are clipped") +
  theme_bw()

```



```

best.radial <- radial.tune$best.model

pred.radial.train <- predict(best.radial, newdata = train1)
pred.radial.test <- predict(best.radial, newdata = test1)

```

```
train_err_svmr <- mean(pred.radial.train != train1$mpg_cat)
test_err_svmr  <- mean(pred.radial.test  != test1$mpg_cat)

confusionMatrix(data = pred.radial.test, reference = test1$mpg_cat)$table
```

```
##           Reference
## Prediction low high
##         low  51   3
##         high   8  56
```

The CV-selected radial-kernel parameters are $\text{cost} = 1.3038$ and $\text{gamma} = 0.6065$. The radial SVM achieves a **training error rate of 0.04** and a **test error rate of 0.093**.

(c) Compare the performance of the above models with boosting. Do SVC or SVM provide better predictive performance on this dataset?

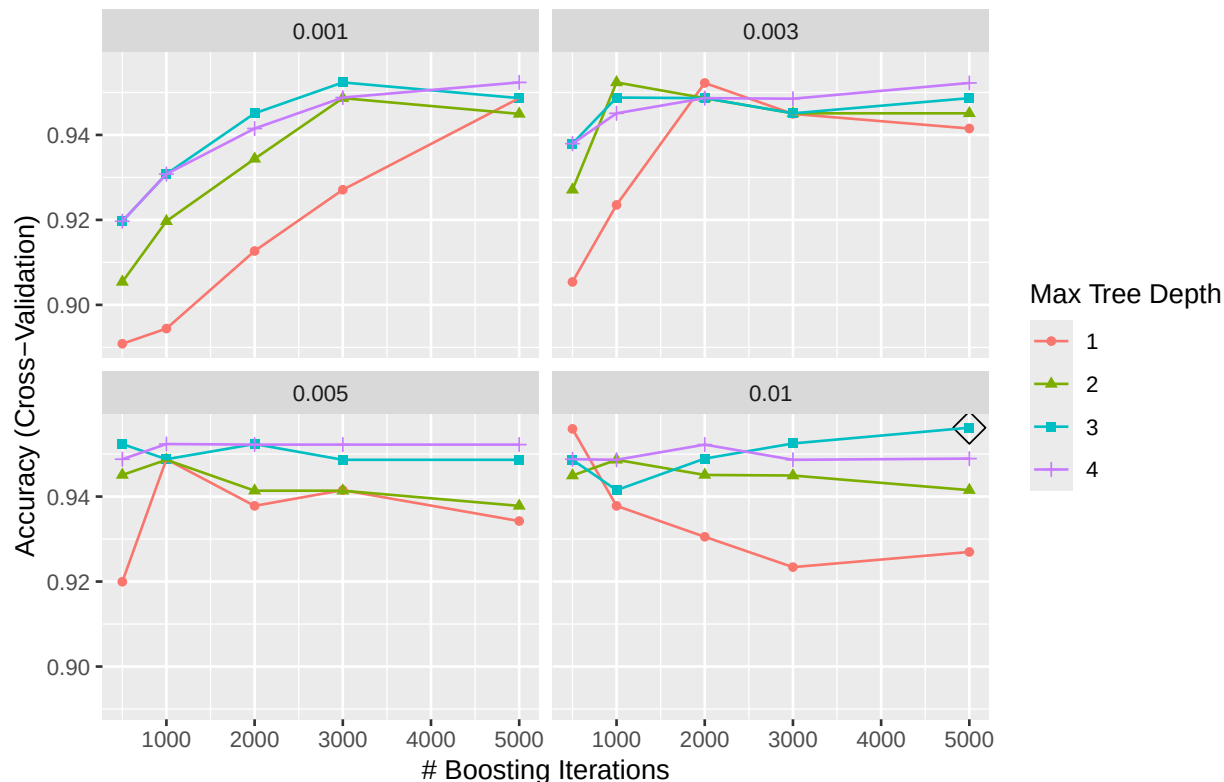
We fit a gradient boosting model on the **same training set** using `caret::train(method = "gbm")` with `distribution = "adaboost"`, tuning over `n.trees`, `interaction.depth`, `shrinkage`, and `n.minobsinnode` (mirroring `gbmA.grid` from Lecture 11).

```
ctrl <- trainControl(method = "cv", number = 10)

gbmA.grid <- expand.grid(n.trees      = c(500, 1000, 2000, 3000, 5000),
                        interaction.depth = 1:4,
                        shrinkage      = c(0.001, 0.003, 0.005, 0.01),
                        n.minobsinnode = 10)

set.seed(2026)
gbmA.fit <- train(mpg_cat ~ .,
                  data      = train1,
                  method    = "gbm",
                  tuneGrid  = gbmA.grid,
                  trControl = ctrl,
                  distribution = "adaboost",
                  verbose    = FALSE)

ggplot(gbmA.fit, highlight = TRUE)
```



```
pred.gbm.train <- predict(gbmA.fit, newdata = train1)
pred.gbm.test  <- predict(gbmA.fit, newdata = test1)

train_err_gbm <- mean(pred.gbm.train != train1$mpg_cat)
test_err_gbm  <- mean(pred.gbm.test  != test1$mpg_cat)
```

```
compare_tbl <- tibble(
  Model      = c("SVC (linear)", "SVM (radial)", "Boosting (AdaBoost)"),
  `Train error` = c(train_err_svc, train_err_svmr, train_err_gbm),
  `Test error`  = c(test_err_svc, test_err_svmr, test_err_gbm)
)
knitr::kable(compare_tbl, digits = 3,
  caption = "Train and test error rates on the shared 70/30 split.")
```

Table 1: Train and test error rates on the shared 70/30 split.

Model	Train error	Test error
SVC (linear)	0.073	0.102
SVM (radial)	0.040	0.093
Boosting (AdaBoost)	0.000	0.068

```
best_idx <- which.min(compare_tbl$`Test error`)
best_model <- compare_tbl$Model[best_idx]
```

Test errors: SVC = 0.102, SVM (radial) = 0.093, Boosting (AdaBoost) = 0.068. **Boosting (AdaBoost)** attains the lowest test error. The radial SVM does edge out the linear SVC by one misclassification (0.093

vs 0.102), but the gap is small. Boosting cuts the test error further to 0.068 – noticeably lower than either SVM. Boosting fits the training data perfectly (train error = 0), but its 10-fold CV accuracy and held-out test error are both the best of the three, so this is a benign training-set memorisation rather than damaging overfitting. **Answer to the question:** neither SVC nor SVM-radial provides better predictive performance than boosting; boosting wins on this dataset.

Problem 2

In this problem, we perform hierarchical clustering on the states using the `USArrests` data in the `ISLR` package.

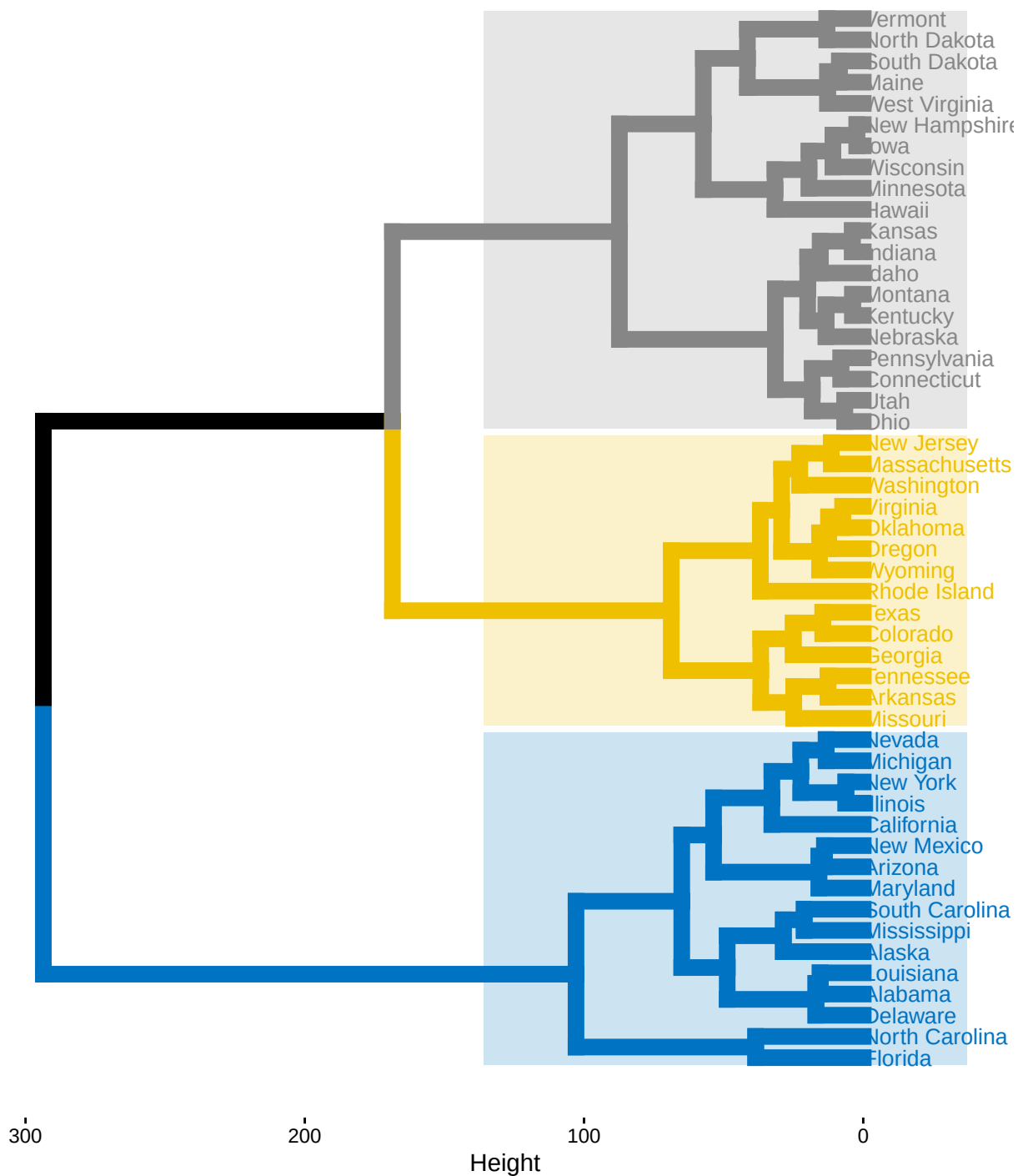
```
data("USArrests")
arrests <- USArrests
```

(a) Using hierarchical clustering with complete linkage and Euclidean distance, cluster the states. Cut the dendrogram at a height that results in three distinct clusters. Which states belong to which clusters?

```
hc.complete <- hclust(dist(arrests), method = "complete")

fviz_dend(hc.complete, k = 3,
  horiz = TRUE,
  cex = 0.7,
  palette = "jco",
  color_labels_by_k = TRUE,
  rect = TRUE, rect_fill = TRUE, rect_border = "jco",
  main = "Complete linkage, unscaled") +
  theme(legend.position = "none")
```

Complete linkage, unscaled



```
cluster_a <- cutree(hc.complete, k = 3)
table(cluster_a)
```

```
## cluster_a
```

```
## 1 2 3
## 16 14 20
```

```
for (k in 1:3) {
  states_k <- names(cluster_a)[cluster_a == k]
  cat(sprintf("**Cluster %d** (n = %d): %s.\n\n",
    k, length(states_k),
    paste(states_k, collapse = ", ")))
}
```

Cluster 1 (n = 16): Alabama, Alaska, Arizona, California, Delaware, Florida, Illinois, Louisiana, Maryland, Michigan, Mississippi, Nevada, New Mexico, New York, North Carolina, South Carolina.

Cluster 2 (n = 14): Arkansas, Colorado, Georgia, Massachusetts, Missouri, New Jersey, Oklahoma, Oregon, Rhode Island, Tennessee, Texas, Virginia, Washington, Wyoming.

Cluster 3 (n = 20): Connecticut, Hawaii, Idaho, Indiana, Iowa, Kansas, Kentucky, Maine, Minnesota, Montana, Nebraska, New Hampshire, North Dakota, Ohio, Pennsylvania, South Dakota, Utah, Vermont, West Virginia, Wisconsin.

(b) Hierarchically cluster the states using complete linkage and Euclidean distance, after scaling the variables to have standard deviation one. Does scaling the variables change the clustering results? In your opinion, should the variables be scaled before the inter-observation dissimilarities are computed?

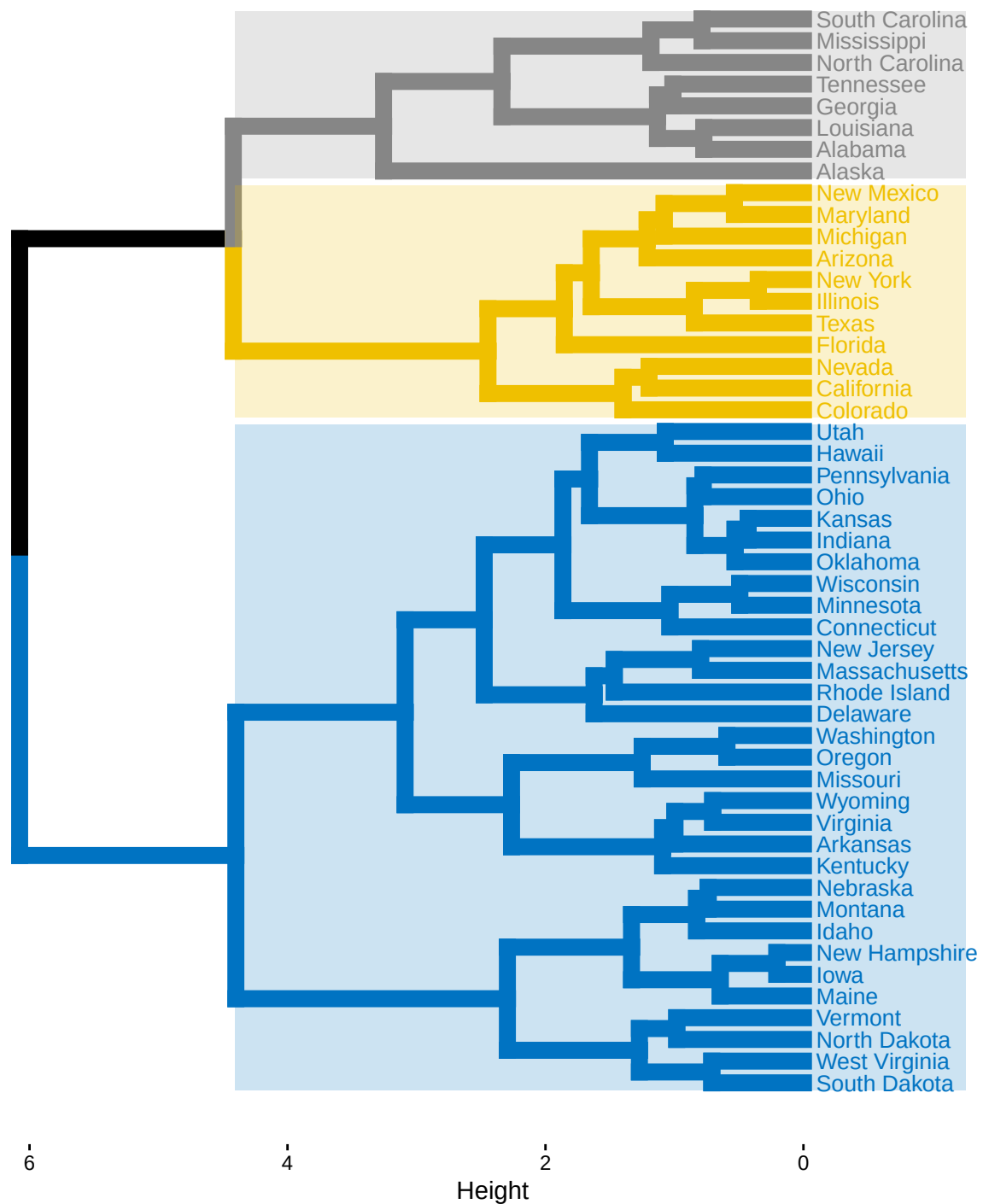
```
apply(arrests, 2, sd)
```

```
## Murder Assault UrbanPop Rape
## 4.355510 83.337661 14.474763 9.366385
```

```
arrests_scaled <- scale(arrests)
hc.complete.s <- hclust(dist(arrests_scaled), method = "complete")

fviz_dend(hc.complete.s, k = 3,
  horiz = TRUE,
  cex = 0.7,
  palette = "jco",
  color_labels_by_k = TRUE,
  rect = TRUE, rect_fill = TRUE, rect_border = "jco",
  main = "Complete linkage, scaled to unit SD") +
  theme(legend.position = "none")
```

Complete linkage, scaled to unit SD



```
cluster_b <- cutree(hc.complete.s, k = 3)
table(cluster_b)
```

```
## cluster_b
```

```
## 1 2 3
## 8 11 31
```

```
table(unscaled = cluster_a, scaled = cluster_b)
```

```
##          scaled
## unscaled 1 2 3
##          1 6 9 1
##          2 2 2 10
##          3 0 0 20
```

Does scaling change the clustering? Yes. The cross-tabulation `table(unscaled, scaled)` is not block-diagonal (e.g. unscaled cluster 1 splits across scaled clusters 1, 2, and 3), so the two clusterings are not just relabellings of each other.

Should the variables be scaled? Yes. The four `USArrests` variables are on very different scales: `Assault` has `sd = 83.34`, while `UrbanPop` (`sd = 14.47`), `Rape` (`sd = 9.37`), and `Murder` (`sd = 4.36`) are an order of magnitude smaller. With unscaled Euclidean distance, the inter-state dissimilarity is dominated almost entirely by `Assault`, so `Murder`, `Rape`, and `UrbanPop` contribute very little. Scaling each variable to unit standard deviation puts the four features on comparable footing, so the dissimilarities reflect all four crime/demographic dimensions – the more defensible choice here.